From Formal Verification to Generative Paradigms:

A Comparative Study of File System Design Methodologies

by

Xiaochuan Yan

225045041

Final Report

Prepared for the Operating Systems course

in partial fulfillment of the course requirements

School of Data Science

The Chinese University of Hong Kong, Shenzhen

April 2026

Abstract

File systems are a cornerstone of modern operating systems, providing critical abstractions for managing persistent data. Their development is difficult because it demands simultaneous reasoning about crash safety, fine-grained concurrency, and long-term feature evolution. An empirical study of Ext4—one of the most deployed Linux file systems—shows that bug fixes and maintenance account for 82.4% of all commits over its two-decade history, while new features constitute only 5.1%. This report examines three papers that address file system correctness through progressively more automated means: FSCQ (SOSP 2015), which introduces Crash Hoare Logic to produce the first certified crash-safe file system; CRL-H and AtomFS (SOSP 2019), which extends formal guarantees to fine-grained concurrent file systems through a helper mechanism for external linearization points; and SysSpec with SpecFS (2025), which replaces hand-written proofs with LLM-based generation guided by formal-method-inspired specifications.

These three works trace a trajectory in file system engineering methodology. The scope of correctness guarantees expands from crash safety to linearizability to specification-level evolvability. The role of formal methods shifts from a post-hoc verification tool to a co-design framework to the primary design artifact from which implementation is derived. The developer's task transitions from writing and proving code to designing specifications. This report applies a unified analytical framework examining each work along five dimensions—correctness target, specification language, verification methodology, performance, and evolvability—to characterize both cumulative progress and persisting trade-offs.

Contents

1 Introduction **5**

1.1 Research Questions 6

1.2 Methodological Posture 7

1.3 Chapter Outline 7

2 Background and Preliminaries **9**

2.1 Why File System Verification Is Difficult 9

2.1.1 Crash Safety 9

2.1.2 Concurrency and Linearizability 9

2.1.3 Evolvability 10

2.2 Key Formal Concepts 10

2.2.1 Hoare Logic and Crash Extensions 10

2.2.2 Separation Logic 10

2.2.3 Rely-Guarantee Reasoning 11

2.2.4 Linearizability and Contextual Refinement 11

2.2.5 Mechanized Proof Assistants 11

2.3 Broader Related Work 11

2.3.1 Testing and Bug Finding in File Systems 11

2.3.2 Verified Systems Software 12

2.3.3 Concurrent Program Verification 12

2.3.4 LLM-Based Code Generation 12

2.3.5 The Broader File System Evolution Landscape 13

2.4 Analytical Framework 13

3 FSCQ: Certified Crash-Safe File System with Crash Hoare Logic **15**

3.1 Motivation and Problem Setting 15

3.2 Crash Hoare Logic: Specification Framework 15

3.2.1 Crash Conditions and Recovery Semantics 16

3.2.2 Logical Address Spaces 16

3.2.3 Crash Conditions and the Asynchronous Disk Model 17

3.2.4 Recovery Execution Semantics 17

3.3 Proof Automation in CHL 17

3.4 FSCQ System Architecture . . . 19
3.5 Implementation and Deployment . . . 20
3.6 Evaluation . . . 21
3.6.1 Correctness Guarantees . . . 21
3.6.2 Performance . . . 21
3.6.3 Proof Effort and Evolvability . . . 22
3.7 Strengths . . . 22
3.8 Limitations . . . 23
3.9 Position in the Evolution of File System Verification . . . 24

4 CRL-H and AtomFS: Verified Concurrent File System with Helper Mechanism 25
4.1 Motivation and Problem Setting . . . 25
4.2 Path Inter-Dependency in Detail . . . 25
4.3 CRL-H: Concurrent Relational Logic with Helpers . . . 27
4.3.1 Overview of the Logic . . . 27
4.3.2 Ghost State and Helper Metadata . . . 27
4.3.3 The `linothers` Primitive and the Helping Procedure . . . 27
4.3.4 The Roll-Back Mechanism . . . 28
4.3.5 Proving with CRL-H . . . 28
4.4 AtomFS System Design . . . 30
4.5 Evaluation . . . 30
4.5.1 Verification Scale . . . 30
4.5.2 Correctness . . . 31
4.5.3 Performance and Scalability . . . 31
4.6 Strengths . . . 31
4.7 Limitations . . . 32
4.8 Position in the Evolution of File System Verification . . . 33

5 SysSpec and SpecFS: A Case for Generative File Systems 34
5.1 Motivation and Problem Setting . . . 34
5.2 The SysSpec Specification Language . . . 35
5.2.1 Functionality Specification . . . 35
5.2.2 Modularity Specification . . . 36
5.2.3 Concurrency Specification . . . 36
5.2.4 DAG-Structured Specification Patches . . . 37
5.3 The SysSpec Toolchain . . . 38
5.4 SpecFS: A Concrete Instantiation . . . 39
5.5 Evaluation . . . 39

5.5.1 Generation Accuracy . . . 39
5.5.2 Evolvability . . . 40
5.5.3 Ablation Study . . . 41
5.5.4 Productivity . . . 41
5.5.5 Performance . . . 41
5.6 Strengths . . . 42
5.7 Limitations . . . 42
5.8 Position in the Evolution of File System Development . . . 43

6 Cross-Paper Synthesis and Open Challenges 44
6.1 Revisiting the Research Questions . . . 44
6.1.1 Correctness Scope . . . 44
6.1.2 Role of Formal Methods . . . 44
6.1.3 Proof Effort, Performance, and Generality . . . 45
6.2 Open Challenges . . . 45

7 Conclusion and Future Work 47

Chapter 1
Introduction

File systems manage persistent data and provide the namespace and access-control mechanisms that applications depend on. Their design is shaped by the characteristics of underlying storage hardware and the demands of evolving application workloads [5, 27, 40]. As hardware capabilities advance and application requirements change, file systems must continuously incorporate new features, optimize I/O paths, and fix correctness bugs. This ongoing evolution makes file system development one of the most demanding areas of systems software.

A quantitative analysis by Liu et al. [27], as shown in Figure 1.1, illustrates the magnitude of this maintenance challenge. Examining all 3,157 Ext4-related commits merged into the Linux kernel from version 2.6.19 through 6.15, the study categorizes each commit as a bug fix, performance optimization, reliability enhancement, feature addition, or maintenance change. The results show that 82.4% of all commits are dedicated to bug fixes and maintenance, while only 5.1% introduce new functionality. Among the bugs, 62.1% are semantic errors involving incorrect logic, compared to 15.4% for memory errors and 15.1% for concurrency errors. Furthermore, the evolution is not monotonic: even after the codebase matures, change rates increase again, peaking at Linux 5.10—more than a decade after Ext4's initial release. This pattern indicates that file systems remain in a state of constant evolution throughout their lifetime.

A case study of the fast-commit feature further illustrates the difficulty of evolving file systems. Fast commit, a hybrid journaling mechanism introduced in Linux 5.10, required 9 commits for its initial implementation (adding over 4,000 lines of code spanning core modules such as inode, file, and journal). However, the feature subsequently required approximately 80 follow-up commits for bug fixes, of which over 65% addressed semantic errors including misordered metadata updates and incorrect handling of corner cases. Each individual change (with LLMs) introduced during the evolution process can potentially trigger cascading effects on other file system modules, particularly those with existing dependencies. For example, a seemingly local feature addition, like introducing extent [12], can trigger non-local changes by altering core data structures like the inode, affecting any module that interacts with it and making manual dependency management intractable.

This report examines three research papers that span a decade of work on improving file system correctness through increasingly automated methodologies:

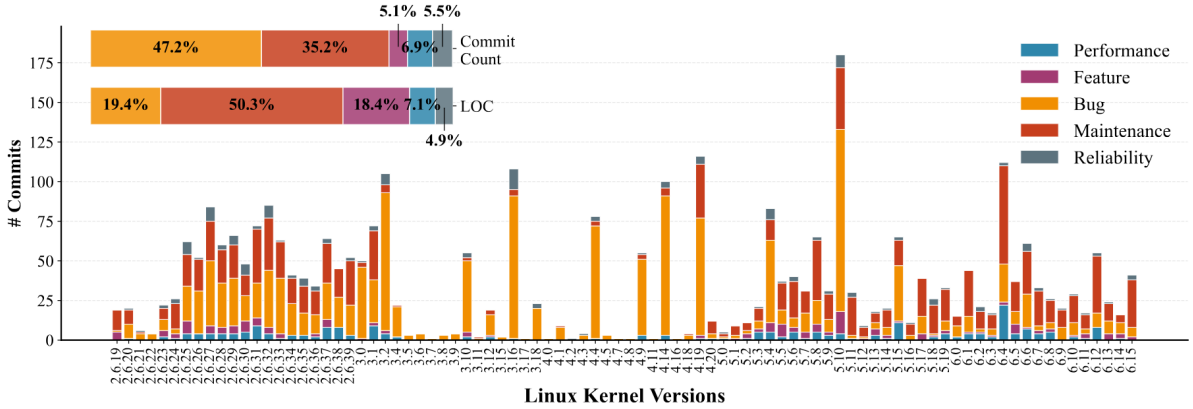

Figure 1.1: File system evolution with different types of patches.

1. **FSCQ** [5] (SOSP 2015) introduces *Crash Hoare Logic* (CHL), a formal framework extending Hoare logic with crash conditions, recovery procedures, and logical address spaces. Using CHL embedded in the Coq proof assistant, the authors develop FSCQ—the first file system with a machine-checkable proof that its implementation correctly recovers from any sequence of crashes. The verified guarantees include no data loss, directory tree acyclicity, and atomicity of system calls such as `rename`.

2. **CRL-H and AtomFS** [40] (SOSP 2019) extends formal verification to the concurrent setting. It identifies *path inter-dependency* as the phenomenon that causes external linearization points in fine-grained concurrent file systems—a problem that affects all nine file systems surveyed by the authors. To address this, they introduce *Concurrent Relational Logic with Helpers* (CRL-H), which incorporates a helper mechanism, ghost state, and a roll-back procedure. They apply CRL-H to build AtomFS, the first formally verified, fine-grained, concurrent file system with linearizable POSIX-like interfaces.

3. **SysSpec and SpecFS** [27] (2025) proposes a paradigm shift: instead of hand-writing both code and proofs, developers write structured, multi-part specifications from which large language models (LLMs) generate correct C code. SysSpec adapts concepts from formal methods—Hoare-logic pre/post-conditions, rely-guarantee contracts, and invariants—into a specification language designed for LLM consumption. The resulting system, SpecFS, passes 690 of 754 xfstests, matches manually-coded baselines, and demonstrates 3–5× productivity improvements for feature additions.

1.1 Research Questions

This review is structured around three research questions:

1. **How has the scope of correctness guarantees expanded across these works?** FSCQ addresses crash safety for a sequential file system; CRL-H adds linearizability under fine-grained concurrency through the helper mechanism; SysSpec addresses correctness during generation (specification conformance) and, crucially, evolvability—the ability to integrate new features without breaking existing invariants.

2. **How has the role of formal methods shifted?** In FSCQ, formal methods serve as a verification tool applied after system design. In CRL-H, the logic's requirements (e.g., the non-bypassable criterion) directly shape the system architecture, making formal reasoning a co-design activity. In SysSpec, formal-method-inspired constructs become the primary specification language from which a toolchain derives implementation, shifting the developer's role from proof-writer to specification-writer.

3. **What trade-offs exist among proof effort, performance, and generality?** Each paper makes distinct choices. FSCQ's proofs are exhaustive but the system is sequential and simplified. AtomFS achieves concurrent verification at the cost of a 94:1 proof-to-code ratio. SpecFS trades formal certainty for generation speed and evolvability, relying on agentic validation rather than mechanized theorems.

1.2 Methodological Posture

This report adopts an interpretive and comparative posture. It does not re-implement the three systems. Instead, it examines the primary papers' formalizations, evaluates the coherence of their claims under a shared set of comparative dimensions, and situates each work within the broader literature on file system correctness and code generation. Quantitative comparisons across papers are treated with care, as the evaluation settings (hardware, workloads, baseline systems) differ. The report's contribution is structured synthesis and critical comparison rather than independent empirical validation.

1.3 Chapter Outline

Chapter 2 presents the technical background on file system verification challenges and the broader related work context, establishing the analytical framework used throughout. Chapters 3, 4, and 5 examine FSCQ, CRL-H/AtomFS, and SysSpec/SpecFS respectively. Each follows a parallel structure: problem setting and motivation, core technical method (with detailed exposition of key mechanisms), system design and implementation, evaluation results across multiple dimensions, strengths and limitations, and position in the

evolutionary trajectory. Chapter 6 provides a cross-paper synthesis organized around the three research questions and identifies open challenges. Chapter 7 summarizes findings.

Chapter 2
Background and Preliminaries

This chapter establishes the technical context for understanding the three focal papers. Section 2.1 characterizes the specific challenges that make file system verification uniquely difficult. Section 2.2 introduces the formal concepts that recur across the papers. Section 2.3 surveys broader related work that contextualizes the three focal systems. Section 2.4 presents the unified analytical framework used for comparison throughout the report.

2.1 Why File System Verification Is Difficult

2.1.1 Crash Safety

Most software is verified under the assumption of continuous, fault-free execution. File systems must additionally guarantee correctness under arbitrary crash-reboot cycles. A crash can expose intermediate on-disk states that were never intended to be observed. For example, a `rename` operation involves multiple disk writes (updating the source directory, the destination directory, and potentially the inode itself). Without careful write ordering enforced by mechanisms such as write-ahead logging, a crash between writes could leave the file in neither directory—a violation of POSIX atomicity semantics [32]. The developer must ensure that, under any sequence of crashes followed by recovery, the file system returns to a consistent state. For example, after recovery, either all disk writes from a file system call will be on disk, or none will be. Pillai et al. [32] demonstrated that even mature applications make inconsistent assumptions about file system crash behavior, underscoring that the problem is both subtle and practically consequential.

2.1.2 Concurrency and Linearizability

Modern file systems employ fine-grained locking to scale on multicore processors. This introduces non-deterministic interleavings of operations that must all produce correct results. The standard correctness criterion is *linearizability* [15]: each operation must appear to take effect atomically at a single instant between its invocation and response, and the resulting sequential history must be legal with respect to the sequential specification.

File systems introduce a unique complication: *path inter-dependency* [40]. Consider two concurrent operations: `mkdir(/a/b/c)` and `rename(/a, /e)`. If `rename` completes

while `mkdir` is mid-traversal, the `mkdir`'s path is broken. For the execution to be linearizable, `rename` must logically "help" `mkdir` commit its effect before `rename` takes effect. This forces the linearization point of `mkdir` to be *external*—it resides in the code of `rename`, not in `mkdir` itself. Standard verification approaches based on fixed, code-local linearization points cannot handle this phenomenon. Zou et al. [40] confirmed through a survey of nine file systems (six Linux, two FreeBSD, xv6fs) that path inter-dependency is universal across all combinations of `rename` with other path-based operations.

2.1.3 Evolvability

File systems are not static artifacts. They accumulate features over decades, as illustrated by Ext4's integration of extents (2006), delayed allocation (2008), metadata checksums (2012), inline data (2013), and encryption (2015) [27]. Each feature addition modifies core data structures (e.g., the inode) and can interact with every module that accesses them. A verification methodology must therefore support incremental change without requiring re-verification of the entire system from scratch.

2.2 Key Formal Concepts

2.2.1 Hoare Logic and Crash Extensions

Hoare logic [16] provides the foundational notation: a triple $\{P\}\ C\ \{Q\}$ states that if precondition P holds before executing command C, then postcondition Q holds afterward. Crash Hoare Logic (CHL), introduced by Chen et al. [5], extends this with a crash condition and a recovery procedure: $\{P\}\ C\ \{Q\}\ [R]$ means that if P holds, then either C completes normally (establishing Q), or a crash occurs during C, and after executing recovery procedure R (possibly multiple times if crashes occur during recovery), the state satisfies Q. This formulation captures the all-or-nothing semantics that file system operations must provide under crash-reboot cycles.

2.2.2 Separation Logic

Separation logic [33] extends Hoare logic with the separating conjunction $\star$, which asserts that a state can be decomposed into disjoint sub-states. This is essential for modular file system verification: the disk can be decomposed into disjoint regions (allocation bitmap, inode table, data blocks), and each layer of the file system can own and reason about its region independently. Both FSCQ and CRL-H employ separation logic for this purpose.

2.2.3 Rely-Guarantee Reasoning

Rely-guarantee (R-G) reasoning [17] provides compositional verification for concurrent systems. Each thread specifies a *rely condition* R (interference it tolerates from other threads) and a *guarantee condition* G (interference it may impose on others). Compositional correctness follows if each thread's guarantee implies the rely conditions of all other threads. CRL-H combines R-G reasoning with relational specifications to prove linearizability, while SysSpec adapts R-G concepts into its modularity specification language for LLM-guided code generation.

2.2.4 Linearizability and Contextual Refinement

Linearizability [15] is a correctness condition for concurrent objects, e.g., concurrent file systems. For all possible clients, if every observable event trace generated from invoking the implementation can also be produced by invoking the corresponding atomic specification (i.e., the implementation has the same effect as an atomic specification), then the object is atomic. This property is also formalized as contextual refinement. Several researchers have proved that contextual refinement is equivalent to linearizability when the specification is atomic [11, 13, 25]. Therefore, atomicity defined in this work is equivalent to linearizability. CRL-H proves linearizability by establishing a simulation relation between the concrete implementation and an atomic abstract specification, using the helper mechanism to handle external linearization points.

2.2.5 Mechanized Proof Assistants

Both FSCQ and AtomFS mechanize their proofs in Coq [9], an interactive theorem prover in which specifications, implementations, and proofs are all written in a single dependently-typed language. The Coq proof checker is a small, trusted kernel that verifies all proofs mechanically, eliminating reliance on human inspection. Coq also supports *extraction*, which compiles verified Coq code to executable languages such as Haskell or OCaml. FSCQ uses extraction to produce a Haskell executable; AtomFS's verified C code is modeled in Coq using a deeply embedded C semantics.

2.3 Broader Related Work

2.3.1 Testing and Bug Finding in File Systems

Before formal verification became practical for file systems, the primary methods for improving correctness were testing and model checking. Yang et al. [38, 39] developed FiSC and eXplode, model checkers that systematically enumerate crash scenarios and file

system operations to detect inconsistencies. These tools found serious bugs in ext3, JFS, and ReiserFS, demonstrating that even mature file systems contain crash-safety errors. Fuzzing approaches, such as Janus [29] and Hydra [19], further explored the input space of file system calls.

These methods are effective at finding bugs but cannot guarantee their absence. A model checker explores a finite subset of the state space; a fuzzer samples from an infinite input space. This limitation motivated the use of formal verification to establish exhaustive guarantees.

2.3.2 Verified Systems Software

The feasibility of end-to-end system verification was established by several landmark projects. The CompCert compiler [21] is a formally verified C compiler with a machine-checked proof of semantic preservation in Coq. The seL4 microkernel [20] is a formally verified operating system kernel using Isabelle/HOL. While seL4 does not include a file system (persistent state is outside the kernel's scope), it demonstrated that large-scale systems verification is achievable. Verdi [36] applies Coq-based verification to distributed systems, providing a framework for reasoning about node failures and network partitions.

Within file systems specifically, DFSCQ [4] extended FSCQ with higher performance and additional features, while SFSCQ [3] added support for concurrency using sequential reasoning. BilbyFS [2] explored layered domain-specific languages for file system verification. The IronFleet system [14] applied a combination of TLA-style state-machine refinement and Hoare-logic verification to distributed systems.

2.3.3 Concurrent Program Verification

Verifying linearizability of concurrent data structures has received extensive attention. Approaches include LiLi [26] (using atomic abstract specifications) and Iris [18] (a concurrent separation logic framework). However, none of these approaches directly address the external linearization point problem caused by path inter-dependency in file systems. This gap specifically motivates the helper mechanism in CRL-H.

2.3.4 LLM-Based Code Generation

Large language models have demonstrated significant capabilities in code generation. Systems such as Codex [7], CodeGen [30], StarCoder [23], and AlphaCode [24] can generate correct functions and small programs from natural-language prompts. GPT-4 [31] and its successors further improve on these capabilities. However, these systems primarily target algorithmic or application-level code. When applied to systems software, they face the

specification semantic gap: natural language cannot precisely capture multi-faceted requirements including functional correctness, non-functional properties, and concurrency control.

Recent work has explored structured prompting and iterative refinement to improve LLM code generation. Self-Debug [8] uses execution feedback to iteratively fix generated code. SWE-Agent [37] applies LLM agents to software engineering tasks including bug fixing. ChatDBG [22] uses LLMs for program debugging. Retrieval-Augmented Generation (RAG) approaches provide additional context to improve generation quality. However, as Liu et al. [27] observe, these methods still fundamentally rely on natural language for intent specification, which limits their precision for systems software.

2.3.5 The Broader File System Evolution Landscape

The challenge of file system evolution extends beyond Ext4. Other file systems face similar dynamics. Btrfs [34], ZFS [1], and XFS [35] all maintain substantial codebases with ongoing feature development and bug fixes. Lu et al. [28] conducted a comprehensive study of file system bugs across Linux, finding that patch distribution patterns are broadly consistent across file systems: semantic bugs dominate, and complexity correlates with bug density. This reinforces the motivation for methodologies that can systematically manage correctness during evolution.

2.4 Analytical Framework

To enable structured comparison across the three focal papers, this report uses a framework with five dimensions:

1. **Correctness target.** The property being guaranteed: crash safety for FSCQ, linearizability for AtomFS, and specification conformance plus evolvability for SpecFS.

2. **Specification language.** The formalism used to express desired properties: CHL triples for FSCQ, CRL-H judgments for AtomFS, and multi-part SysSpec specifications for SpecFS.

3. **Verification/generation methodology.** How correctness is established: Coq mechanized proofs for FSCQ and AtomFS, LLM-based generation with agentic validation for SpecFS.

4. **Performance characteristics.** The runtime cost of correctness: CPU overhead, I/O behavior, and scalability.

5. **Proof/generation effort and evolvability.** Human effort required and the ability to accommodate new features without re-verification.

Each of the following three chapters applies this framework to one focal paper, and Chapter 6 synthesizes the comparison.

Chapter 3
FSCQ: Certified Crash-Safe File System with Crash Hoare Logic

3.1 Motivation and Problem Setting

File systems have a long history of bugs that cause data loss, particularly around crash handling. Common bug classes include performing disk writes without sufficient ordering barriers, forgetting to zero out newly allocated directory or indirect blocks, and failing to maintain structural invariants such as directory tree acyclicity [5]. Even mature file systems in the Linux kernel contain such bugs during normal operation and in crash recovery [32, 38].

Prior approaches to building crash-safe file systems fall into three categories [5]: testing (including fuzz testing), program analysis, and model checking. Model checking tools such as FiSC [39] and eXplode [38] systematically enumerate crash scenarios and check for inconsistencies. These methods have found serious bugs in ext3, JFS, and ReiserFS. However, they explore only a finite subset of the state space and cannot guarantee the absence of bugs. As Yang et al. [38] note, "the checks are only as good as the checker's specification of correct behavior."

FSCQ directly addresses this limitation. Its goal is to produce a file system with a machine-checkable proof, verified in the Coq proof assistant, that its implementation meets its specification and that the specification includes crash behavior. The proof guarantees that, under any sequence of crashes followed by reboots, the file system will recover to a state consistent with POSIX semantics.

3.2 Crash Hoare Logic: Specification Framework

The central intellectual contribution of FSCQ is *Crash Hoare Logic* (CHL), a specification framework that extends traditional Hoare logic with three key innovations: crash conditions, recovery procedures, and logical address spaces. CHL is implemented as a shallow embedding in Coq, meaning that CHL specifications, implementations, and proofs all reside within the same Coq framework.

3.2.1 Crash Conditions and Recovery Semantics

In traditional Hoare logic, a specification $\{P\}\ C\ \{Q\}$ states that executing C from a state satisfying P yields a state satisfying Q, if C terminates. CHL extends this to:

$$\{P\}\ C\ \{Q\}\ [R]_{\text{rec}} \tag{3.1}$$

where R is the recovery procedure that runs after every crash. The semantics is: if P holds before C executes, then either (a) C completes normally and Q holds (the completed state), or (b) a crash occurs during C, and after executing R—possibly multiple times if crashes occur during recovery itself—the system reaches a state satisfying Q (the recovered state).

The formal execution model uses a "crash-recovery loop": the normal procedure attempts to execute; if the system crashes at any point, the recovery procedure runs; if recovery itself crashes, it restarts. This continues until either the normal procedure finishes (completed state) or the recovery procedure finishes (recovered state). The specification distinguishes these outcomes using a special `ret` variable, enabling the specification to state stronger properties for completion than recovery.

3.2.2 Logical Address Spaces

CHL generalizes separation logic's points-to relation from the physical disk to higher-level abstractions. Instead of specifying every disk block, developers define *logical address spaces* that capture abstractions at different layers:

- The physical disk address space maps block numbers to block contents.
- The transaction/log address space maps block numbers to single values (not sets), exporting a synchronous interface on top of the asynchronous physical disk.
- The inode layer address space maps inode numbers to inode structures.
- The file address space maps logical offsets to data blocks.
- The directory address space maps file names to inode numbers.

A representation invariant, such as `log_rep`, connects a logical address space to its physical representation. For example, `log_rep(NoTxn, start_state)` specifies that the disk is in a state where no transaction is active and that the logical contents match a given address space. The representation invariant for an active transaction, `log_rep(ActiveTxn, start_state, cur_state)`, specifies that the commit block contains zero, all blocks of the starting state are synced to disk, and the current logical state is the result of replaying the in-memory log starting from the starting state.

Logical address spaces address a critical proof-engineering challenge: they enable the logging layer to export a simple, synchronous-disk abstraction to higher layers, so that developers of the inode or directory layer do not need to reason about asynchronous disk writes, write reordering, or partial sector writes.

3.2.3 Crash Conditions and the Asynchronous Disk Model

CHL's disk model captures the behavior of real disks with write buffering and reordering. A disk block is modeled not as a single value but as a tuple $\langle v, vs \rangle$, where v is the last-written value and vs is a set of previous values, any of which could appear on disk after a crash. Reading returns the last-written value, which is correct during normal operation. A sync operation waits until the last-written value is durably on disk and discards the previous-value set.

A crash condition describes the disk state *before* the crash rather than after. This design choice makes crash conditions structurally similar to pre- and postconditions (both describe deterministic states), which simplifies proof automation. The specification of `disk_write` (Figure 3.1) illustrates this: the crash condition states that either the old value or the new value *could* be on disk after a crash, expressed as a disjunction over the possible states before the crash.

3.2.4 Recovery Execution Semantics

Recovery is not an afterthought in CHL; it is an integral part of each procedure's specification. The $\gg$ operator composes a normal procedure with a recovery procedure. For an operation like `atomic_two_write`, which writes two blocks inside a transaction, the specification with recovery states:

- If the procedure finishes without crashing, both blocks were updated.
- If one or more crashes occurred (with recovery running after each), either both blocks were updated or neither was—achieving the desired all-or-nothing atomicity.

The recovery procedure itself has a CHL specification with its own crash condition, enabling reasoning about crashes during recovery. For a logging system, the recovery procedure is `log_recover`, which checks for a commit record: if present, it completes the transaction by copying log contents to their final locations; if absent, it discards the log.

3.3 Proof Automation in CHL

A practical concern in mechanized verification is the proof burden on developers. CHL addresses this through automated chaining of pre- and postconditions. The proof strategy

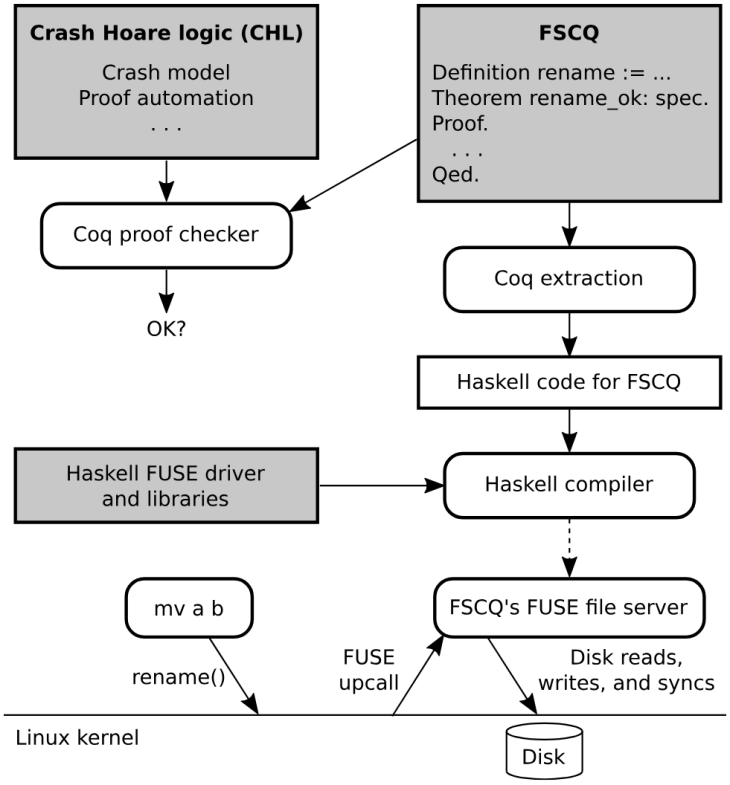

Figure 3.1: Overview of FSCQ's implementation. Rectangular boxes denote source code; rounded boxes denote processes. Shaded boxes denote source code written by hand. The dashed line denotes the Haskell compiler producing an executable binary for FSCQ's FUSE file server.

operates in two phases:

Phase 1: Procedure steps. CHL decomposes the verification of a procedure into proof obligations for each step (procedure call). For each step, it generates two obligations: (1) that the current condition implies the step's precondition, and (2) that the step's crash condition implies the caller's crash condition. This decomposition follows the control flow of the procedure exactly.

Phase 2: Predicate implications. Many obligations generated in Phase 1 are trivial (identical pre- and postconditions can be immediately proved). For more complex cases, CHL uses separation logic to match and cancel predicates. For instance, given an obligation to prove $(a_1 \mapsto v_x \star a_2 \mapsto v_y \star \mathit{others}) \implies (a_1 \mapsto v_0 \star \mathit{other\_blocks})$, CHL matches $a_1 \mapsto v_x$ with $a_1 \mapsto v_0$ (setting $v_0 = v_x$), cancels these terms, and sets $\mathit{other\_blocks}$ to $a_2 \mapsto v_y \star \mathit{others}$. This both proves the obligation and carries information about a_2 forward to subsequent proof steps.

Obligations that CHL cannot prove automatically—typically those involving multiple levels of abstraction, such as connecting a directory abstraction (a function from names to inode numbers) to its concrete representation (directory entries in a file)—require manual developer input. In practice, these correspond to the representation invariants that define

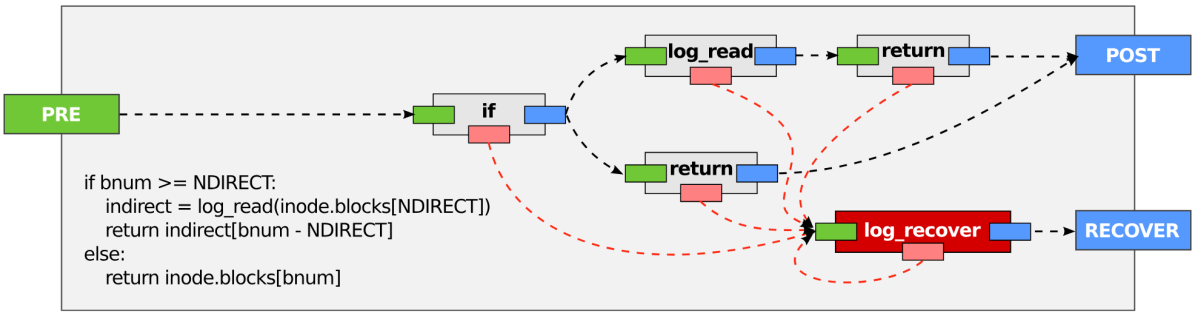

Figure 3.2: Example control flow of a CHL procedure that looks up the address of a block in an inode, with support for indirect blocks. (The actual code in FSCQ checks for some additional error cases.) Gray boxes represent the specifications of procedures. The dark red box represents the recovery procedure. Green and pink boxes represent preconditions and crash conditions, respectively. Blue boxes represent postconditions. Dashed arrows represent logical implication.

each layer's abstraction.

To get some intuition for how CHL can help automate proofs, consider the control flow of the example procedure in Figure 3.2. The outer box corresponds to the top-level specification of a procedure; in this example, it is a procedure that returns the address of the bnumth block from an inode, with recovery-execution semantics. It has a precondition, a postcondition, and a recovery condition.

3.4 FSCQ System Architecture

FSCQ is based on the xv6 file system [10], a teaching operating system implementing Unix v6 semantics with write-ahead logging. Its architecture is organized as a strict layered stack:

FscqLog — Write-ahead logging layer, provides atomic transactions

↓

Buffer Cache — Caches disk blocks in memory

↓

Balloc — Bitmap-based allocator for disk blocks and inodes

↓

Inode Layer — Inode allocation, reading, writing, and indirection

↓

BFile — Block-level file operations (read/write blocks within a file)

↓

Dir — Directory operations (lookup, insert, delete entries)

↓

DirTree — Directory tree consistency (acyclicity, unique parents)

↓

FS — POSIX system call interface (`create`, `read`, `write`, `rename`, etc.)

Each layer uses *representation invariants* that decompose the disk state into disjoint pieces using separation logic's star operator. For example, the file layer invariant splits the disk into four disjoint regions: the allocation bitmap, the inode table, blocks currently in use by files, and free blocks. This decomposition is the foundation of modularity: a proof about the inode layer can assume the bitmap layer maintains its invariants, without inspecting the bitmap's internal code.

The invariant pattern enables incremental evolution. When a layer's implementation changes, only proofs that depend on its representation invariant need to be updated. Layers above that rely on the invariant's interface (but not its implementation) are unaffected. The authors validated this through three case studies: adding asynchronous disk writes (∼1,000 lines changed in FscqLog only), adding indirect block support (∼1,500 lines in the inode layer only), and modifying the buffer cache (∼900 lines across the system, as caching touches multiple layers).

3.5 Implementation and Deployment

FSCQ is implemented in Coq (specifications, implementation, and proofs) and deployed via extraction to Haskell. The extracted Haskell code runs as a FUSE user-level file server, allowing unmodified Unix applications to use FSCQ. The trusted computing base includes Coq's proof checker, the Haskell compiler, the Haskell FUSE library, and the FUSE kernel interface—but not the file system logic itself, whose correctness is machine-checked.

FSCQ supports a subset of POSIX including `create`, `read`, `write`, `link`, `unlink`, `mkdir`, `rmdir`, `rename`, `truncate`, and `stat`. It supports large files via indirect blocks

and nested directories. Notably absent are multiprocessor support, deferred durability (`fsync`), extended attributes, and access control lists—reflecting the design goal of matching xv6's feature set.

3.6 Evaluation

3.6.1 Correctness Guarantees

FSCQ's mechanized proof in Coq eliminates three classes of bugs that have historically caused data loss in file systems:

1. **No data loss:** The allocator's invariants guarantee that each disk block and inode is either allocated or free, never both and never neither. No resource leak is possible, even under arbitrary crash sequences.

2. **Structural consistency:** The directory tree is always acyclic, and every non-root directory has exactly one parent. This prevents "orphaned" directories and infinite traversal loops.

3. **Operation atomicity:** System calls such as `rename` either fully take effect or completely roll back after a crash. No intermediate state (e.g., "file renamed in one directory but not the other") can persist.

The authors validated these guarantees empirically through systematic crash testing. They generated workloads that produce 320 distinct crash states by taking every prefix of writes up to synchronization points combined with subsets of unsynchronized writes. After recovery, FSCQ produced only 10 distinct logical states, all of which were consistent with POSIX semantics (operation either completed fully or rolled back completely). This experimental validation complements the formal proof, providing confidence that the model accurately captures real crash behavior.

3.6.2 Performance

FSCQ's performance is comparable to its unverified xv6 baseline. Compiling the xv6 source tree takes 3.7 seconds on both FSCQ and xv6 when running on an SSD. On a hard disk, FSCQ takes 20 seconds versus 18 seconds for xv6. Compared to Ext4 with data journaling (the most comparable journaling mode), FSCQ is approximately 2× slower (3.7s vs. 2.2s for xv6 compilation), but this gap is attributed to design choices—FSCQ performs 4 disk synchronizations per commit versus Ext4's 2—rather than verification overhead. Ext4 additionally uses optimizations such as asynchronous journal commits that were not implemented in FSCQ.

A notable cost is CPU overhead: FSCQ exhibits roughly 4× CPU overhead compared to xv6's C implementation. This is an artifact of Haskell extraction from Coq (the extracted code uses purely functional data structures and is not optimized for CPU efficiency) rather than a fundamental limitation of CHL.

Table 3.1: Application benchmark performance of FSCQ compared to xv6 and Ext4 configurations. Running time in seconds; lower is better. All file systems are run in synchronous (data-committing) mode except where noted.

Workload	**FSCQ**	**xv6**	**Ext4 (journal)**	**Ext4 (async)**
Git clone	21.0	18.0	13.5	8.0
Make xv6	3.7	3.7	2.2	1.5
Make LFS	5.0	4.5	3.0	2.0
Large file	3.5	3.2	2.5	1.8
Small file	2.8	2.5	2.0	1.5
Cleanup	4.5	4.0	3.0	2.2
Mailbench (msec/msg)	118	103	85	60

3.6.3 Proof Effort and Evolvability

The FSCQ codebase comprises approximately 30,000 lines of Coq (including CHL infrastructure, specifications, implementations, and proofs) compared to approximately 3,000 lines of C for the original xv6—a roughly 10:1 proof-to-code ratio. The breakdown across components shows that the majority of proof effort is concentrated in the more complex layers (logging and directory tree consistency).

The authors demonstrated that FSCQ's modular proof architecture supports incremental evolution. Three case studies showed that localized changes require only localized proof updates: asynchronous disk writes affected only FscqLog; indirect blocks affected only the inode layer; buffer cache changes propagated across the system but still required only 900 lines of total changes. This suggests that the initial proof investment can be amortized over subsequent changes.

3.7 Strengths

1. **End-to-end guarantee.** FSCQ provides the strongest form of correctness guarantee: a machine-checked proof covering the full stack from POSIX system calls

Table 3.2: Combined lines of code and proof for FSCQ components. Total is 31,397 lines covering the CHL infrastructure, specifications, implementations, and proofs.

Component	**Lines of Code**
Fixed-width words	2,691
CHL infrastructure	5,835
Proof automation	2,304
On-disk data structures	7,572
Buffer cache	660
FscqLog (write-ahead logging)	3,163
Bitmap allocator	441
Inodes and files	2,930
Directories	4,489
FSCQ's top-level API	1,312
Total	31,397

to disk writes, including crash-recovery behavior. No prior file system achieved this.

2. **Modular proof architecture.** The layered design, combined with representation invariants expressed in separation logic, enables independent verification of each component. This is essential for scaling formal methods beyond toy examples.

3. **Proof automation.** CHL's two-phase proof strategy significantly reduces the mechanical burden on developers, automating the chaining of specifications along control flow so that manual effort is focused on representation invariants.

4. **Specification clarity.** The formal POSIX specification with crash semantics has independent value: prior work showed that application developers often misunderstand file system crash properties, and FSCQ's specification provides a precise reference.

3.8 Limitations

1. **Sequential execution model.** FSCQ assumes atomic operations and sequential execution. It provides no guarantees for multiprocessor concurrency, which limits applicability to modern multicore systems.

2. **Simplified feature set.** FSCQ targets xv6's feature set, which is considerably simpler than production file systems such as Ext4. Missing features include `fsync`, extended attributes, ACLs, and advanced allocation strategies.

Table 3.3: Incremental evolution case studies in FSCQ, showing that localized changes require only localized proof updates.

Change	**Lines Changed**	**Scope**	**Affected Components**
Asynchronous disk writes	∼1,000	FscqLog only	Modified the logging layer's disk model; layers above unchanged.
Indirect block support	∼1,500	Inode layer	Changed Inode and BFile (50 lines for hard-coded constant); other layers unaffected.
Buffer cache addition	∼900	Cross-system	FscqLog (∼300 lines) and layers above (∼600 lines) to pass cache state.

3. **Proof-to-code ratio.** The roughly 10:1 ratio of proof to implementation code represents a substantial investment. While the authors argue this is manageable for a small research team, it remains a barrier to broader adoption.

4. **Trusted computing base.** The Coq proof checker, Haskell compiler, Haskell runtime, FUSE library, and Linux kernel are all trusted components. A bug in any of these could compromise the extracted system despite verified Coq code.

3.9 Position in the Evolution of File System Verification

FSCQ established the feasibility of end-to-end crash-safety verification for file systems. Its key architectural insight—layered design with separation logic invariants—directly influenced subsequent work on concurrent verification (CRL-H) and specification-guided generation (SysSpec). At the same time, its sequential nature and the substantial proof effort highlight two challenges that animate the later papers: extending guarantees to concurrent settings and reducing the human cost of verification.

Chapter 4
CRL-H and AtomFS: Verified Concurrent File System with Helper Mechanism

4.1 Motivation and Problem Setting

FSCQ demonstrated that formal verification of a sequential, crash-safe file system is feasible. However, real-world file systems must exploit multicore concurrency to achieve scalable performance. Verifying concurrent file systems introduces a fundamentally harder problem: *linearizability*. Each concurrent operation must appear to take effect atomically at a single instant, despite being implemented through multiple fine-grained, interleaved steps.

Zou et al. [40] observe that file systems exhibit a phenomenon that breaks the standard approach to proving linearizability. The standard method identifies each operation's *linearization point* (LP)—the program instruction where the operation's effect becomes atomically visible to other threads. However, file systems exhibit *path inter-dependency*, in which one operation's correctness depends on the state of inodes that another concurrent operation may modify. Specifically, an operation like `rename` may alter a directory structure that other operations are currently traversing, breaking their path integrity. To produce a legal sequential history, the affected operations must appear to complete (and fail) *before* the `rename` takes effect. But this ordering requires the LP of the affected operations to be *external*—it resides in the `rename`'s code, not in the affected operations' own code.

The authors conducted a systematic investigation of nine file systems: six Linux file systems (ext2, ext4, Btrfs, XFS, ReiserFS, Tmpfs), two FreeBSD file systems (UFS, ZFS), and the xv6 teaching file system. Across all nine, and across all five combinations of `rename` paired with other path-based operations (`create`, `unlink`, `mkdir`, `rmdir`), path inter-dependency was observed. This confirms that external LPs are a generic and fundamental phenomenon in concurrent file systems.

4.2 Path Inter-Dependency in Detail

Consider the simplified file system in Figure 4.1, which implements six POSIX interfaces using per-inode locking. Three representative operations illustrate the problem:

```
1  /*error and corner cases
2  handling omitted*/
3  def ins(path, name)
4     cur=root;
5     //traverse path from root
6     ...
7     lock(cur);
8     node=init();
9     insert(cur, name, node);
10    ▶ intuitive LP: INS ◀
11    unlock(cur);
12    return success;
13
14 def del(path, name)
15    cur=root;
16    //traverse path from root
17    ...
18    lock(cur);
19    node=lookup(cur, name)
20    lock(node);
21    delete(cur, name);
22    ▶ intuitive LP: DEL ◀
23    unlock(cur);
24    free(node)
25    return success;
26
27 def rename(src,sn,dst,dn)
28    //traverse path from root
29    //get src directory (sdir)
30    //and dst directory (ddir)
31    ...
32    lock(sdir);
33    lock(ddir);
34    dnode=lookup(ddir,dn);
35    snode=lookup(sdir,sn);
36    lock(dnode);
37    lock(snode);
38    delete(ddir,dn);
39    delete(sdir,sn);
40    insert(ddir,dn,snode);
41    ▶ intuitive LP: RENAME ◀
42    unlock(snode);
43    unlock(sdir);
44    unlock(ddir);
45    free(dnode)
46    return success;
```

Figure 4.1: Pseudocode of a simplified file system. The detailed optimizations, error handling, path traversal code, and stat implementation are omitted for simplicity.

- `ins(path, name)`: Traverses the directory tree, acquires the target directory's lock, inserts a new entry, and releases the lock. The intuitive LP is at the insertion point (before unlock).
- `del(path, name)`: Traverses, acquires the target directory's lock and the target inode's lock, deletes the entry, and releases locks. The intuitive LP is at the deletion point.
- `rename(src, sn, dst, dn)`: Traverses to both source and destination directories, acquires locks on both directories and their target inodes, performs the deletion and re-insertion, and releases locks. The intuitive LP is after all modifications complete.

Now consider concurrent execution of `ins(/a/c)` and `rename(/, a, /, e)`. The `rename` changes the name of directory `/a` to `/e` while the `ins` is trying to create `c` inside `/a`. Following the temporal order of the intuitive LPs, the sequential history would be `rename` then `ins`—but the `rename` breaks `ins`'s path, so `ins` should fail. The only legal sequential history is `ins` (failing) then `rename` (succeeding). This requires `rename` to logically "help" `ins` complete (with failure) before `rename` executes its own LP. The `ins`'s LP is thus external, located within `rename`'s code.

A more complex case is *recursive path inter-dependency*. If three threads execute `rename(/a, /b)`, `rename(/b, /c)`, and `stat(/a/x)` concurrently, the helping relationship chains transitively: the first `rename` must help the `stat` (because `stat` traverses through `/a`), but the second `rename` may also affect the first `rename`'s namespace. The helping set must include not only directly affected operations but also transitively dependent ones.

4.3 CRL-H: Concurrent Relational Logic with Helpers

4.3.1 Overview of the Logic

CRL-H builds on prior work that combines *relational specifications* (which relate concrete states to abstract states) with *local rely-guarantee reasoning* (for compositional concurrency verification). Its novel contribution is the *helper mechanism*, which extends this combination to handle external linearization points.

At the abstract level, a set of *abstract operations* (Aops) specify the desired atomic behavior of the file system. For example, abstract `INS` atomically adds an entry to a directory; abstract `RENAME` atomically moves an entry between directories. At the concrete level, the C implementation executes as fine-grained, interleaved steps. The goal is to prove that every concrete execution is *equivalent* to some sequential execution of abstract operations—i.e., the concrete implementation refines the atomic specification.

4.3.2 Ghost State and Helper Metadata

The helper mechanism relies on *ghost state*—auxiliary variables present in the abstract model but not in the concrete program. Ghost state does not affect runtime behavior; its sole purpose is to enable the proof. CRL-H's ghost state consists of:

- **ThreadPool:** A mapping from thread identifiers to pairs of (Aop_state, Descriptor). The Aop_state is either `(aop, args)` (the abstract operation to be executed) or `(end, ret)` (the operation has finished with return value `ret`).
- **HelpList:** Records the order in which threads received help. This order is used to determine the sequential history for linearizability.
- **Descriptor:** Per-thread metadata including an `Effect` field that records which abstract inodes were modified and how, enabling the roll-back mechanism.

4.3.3 The `linothers` Primitive and the Helping Procedure

The helper mechanism is implemented through a new primitive, `linothers`(t), invoked before thread t executes its own LP. The procedure:

1. Examines the ghost state to identify all threads whose path integrity would be broken by t's operation (using the *linearize-before* relation).
2. Determines a correct helping order, handling recursive dependencies transitively.
3. Executes the abstract operations of all helped threads atomically at the abstract level, recording their effects in ghost state.

4. Updates the HelpList to reflect the order of helped operations.

The *linearize-before* relation defines which operations must precede others in the sequential history. It is derived from the ghost state, which records each thread's traversal history (the inodes it has accessed along its path). If thread t's operation would modify an inode that another thread u's path depends on, then u is linearize-before t. The relation must be transitively closed to handle recursive dependencies.

4.3.4 The Roll-Back Mechanism

When a helper executes abstract operations early, the abstract state moves ahead of the concrete state. The helped operations are still physically running (their concrete code has not yet completed), so the abstract and concrete states diverge. To reconcile this divergence, CRL-H employs a *roll-back mechanism* rather than forcing the concrete state forward.

The key idea: the effects of helped abstract operations are recorded in each thread's `Effect` field in ghost state. When establishing the *abstraction relation* that connects concrete to abstract state, these effects are rolled back from the abstract state. Formally:

$$\exists\, \mathit{effects}, \mathit{Ino}'.\ \mathit{Ino}' = \mathit{rollback}(\mathit{Ino}, \mathit{effects}) \wedge \mathit{match}(\mathit{ino}, \mathit{Ino}') \tag{4.1}$$

where $\mathit{effects} = \mathit{search}(\mathit{inum}, \mathit{ThreadPool}, \mathit{HelpList})$ collects all effects on the given inode number, ordered in reverse HelpList order (last-applied effect is rolled back first). The `rollback` function applies these effects in reverse to the abstract inode Ino, producing Ino', which is then compared to the concrete inode ino.

This formulation is concise because abstract-state manipulation is simpler than concrete-state manipulation. It avoids the need to modify the running concrete code of helped threads.

4.3.5 Proving with CRL-H

The CRL-H judgment takes the form:

$$R; G; I \vdash \{P \star (\mathit{aop}, \mathit{args})\}\ C\ \{Q \star (\mathit{end}, \mathit{ret})\} \tag{4.2}$$

Invariants (I) specify the well-formedness of shared states as relational specifications over concrete state, abstract state, and ghost state, using separation logic's $\star$ to assert disjointness of state components. For instance:

$$I(\mathit{cfs}, \mathit{afs}, \mathit{ghost}) = \mathit{cinv}(\mathit{cfs}) \star \mathit{ainv}(\mathit{afs}) \star \mathit{ginv}(\mathit{ghost}) \wedge \mathit{acrel}(\mathit{cfs}, \mathit{afs}) \tag{4.3}$$

Table 4.1: Key invariants used in AtomFS verification. Each invariant specifies restrictions across abstract FS, concrete FS, and ghost state.

Invariant	**States specified**	**Description**
Abstract-concrete-relation	Abstract FS and Concrete FS	Concrete inode and abstract inode are related using rollback mechanism.
Helped-non-bypassable	Ghost state	A helped Op cannot bypass another Op helped before it.
Unhelped-non-bypassable	Ghost state	An unhelped Op cannot bypass a helped Op.
GoodAFS	Abstract FS	The abstract FS has tree properties like root reachability.
Last-locked-lockpath	Concrete FS and Ghost state	The last inode in the LockPath of thread t is locked by t in concrete FS.
Helplist-consistency	Ghost state	An abstract operation is helped iff its thread ID is in the Helplist.
Future-lockpath-validness	Abstract FS and Ghost state	Thread t will acquire locks indicated by FutLockPath.
Lockpath-wellformed	Ghost state	The LockPathPrefix relation is acyclic.

where *cinv* describes the concrete data layout, *ainv* constrains the abstract state to form a valid file system (e.g., directory tree), *ginv* constrains ghost state, and *acrel* is the abstraction relation.

Rely (R) and guarantee (G) conditions specify the allowed transitions on shared states. A thread's guarantee specifies what changes it may make; its rely is the union of all other threads' guarantees. Compositional verification follows: each thread's guarantee must imply the rely conditions of all other threads.

CRL-H provides two kinds of inference rules:

- **C-statement rules** (SequenceRule, IfRule, WhileRule, AtomRule) for stepping through C code, similar to standard separation logic.
- **Implication rules** for updating abstract and ghost state, including a `LinothersRule` that performs helping on states as described above. Auxiliary commands like `linothers` are removed during verification (they exist only in the abstract model).

A soundness theorem establishes that if each concrete operation can be proved against its abstract specification under CRL-H, then the file system implementation is atomic

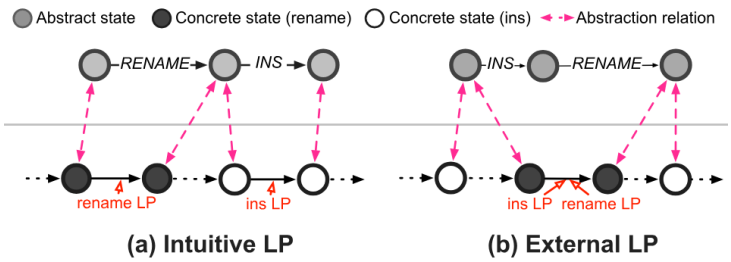

Figure 4.2: Simulation diagrams. The central line is the division of the abstract level (above) and concrete level (below). Figure(a) shows the simulation diagram when an operation's LP resides in its own implementation. In Figure(b), the LP of ins is external and resides in rename's code.

(linearizable). The proof uses a simulation argument: CRL-H judgments establish a simulation relation between concrete and abstract executions, and prior work has established that simulation implies contextual refinement, which is equivalent to linearizability when the specification is atomic [13]. An example is shown in Figure 4.2.

4.4 AtomFS System Design

AtomFS is a concurrent in-memory file system running on FUSE, designed specifically to be verifiable with CRL-H. Its key design principle is the *non-bypassable criterion*, enforced through *lock coupling*: during path traversal, a thread holds the lock of each inode until it acquires the lock of the next inode in the path. This prevents concurrent operations from "bypassing" each other—if thread A bypasses thread B (acquires a lock on a descendant inode before B does), it can produce results that cannot be explained by any sequential history.

Lock coupling ensures that all operations traverse the tree in a consistent order relative to each other. While this may reduce concurrency compared to lock-free designs, it makes the system's behavior provably linearizable under CRL-H.

AtomFS supports six POSIX interfaces: `mknod`, `mkdir`, `rmdir`, `unlink`, `rename`, and `stat`. For verification, `mknod`/`mkdir` are treated as a unified `ins` operation and `unlink`/`rmdir` as `del`. The file system uses per-inode locks for fine-grained concurrency.

4.5 Evaluation

4.5.1 Verification Scale

All proofs for AtomFS are mechanized in Coq. The verification effort is substantial: approximately 63,000 lines of specification and proof code to verify 673 lines of C implementation (excluding comments and blank lines). The breakdown is: 344 lines for abstract operations and abstraction definitions, 1,397 lines for invariants, 451 lines for

rely-guarantee conditions, and the remainder for lemmas and proof scripts. This corresponds to roughly a 94:1 proof-to-code ratio, reflecting the inherent difficulty of concurrent verification.

Table 4.2: Lines of specifications, implementations, and proofs for AtomFS, broken down by component.

Component	**Lines of Code/Proof**
Abstraction and Abstract Operations (Aops)	344
Invariants	1,397
Rely-Guarantee conditions	451
Verified C code (implementation)	673
Proof scripts and lemmas	60,324
Total	63,099

4.5.2 Correctness

AtomFS passed 418 out of 451 test cases in the Linux `xfstests` suite. All 33 failures were attributed to intentionally unimplemented features rather than correctness bugs, providing empirical validation consistent with the formal proof.

4.5.3 Performance and Scalability

AtomFS demonstrates that formal verification and performance are not mutually exclusive. On application workloads including git operations, compilation, file copy (`cp`), and ripgrep, AtomFS with fine-grained locking outperforms a big-lock variant. For the `git` workload, AtomFS achieves roughly 2–3× speedup over the big-lock version. Under small-file workloads (10K files of 1KB each, representing metadata-intensive operations) and large-file workloads (10MB file, representing data-intensive operations), AtomFS's throughput scales with core count, while the big-lock version shows no scalability and Ext4 shows moderate scaling.

4.6 Strengths

1. **Addresses a fundamental challenge.** The helper mechanism directly tackles external linearization points, a phenomenon confirmed to exist in all nine surveyed file systems. This establishes CRL-H as broadly applicable to concurrent file system verification.

Table 4.3: Application workload performance comparison. Running time in seconds for each workload across AtomFS and baselines. Lower is better.

Workload	**AtomFS**	**AtomFS-biglock**	**Tmpfs**	**Ext4**
Large file (10 MB)	2.5	4.5	1.5	1.2
Small file (10K×1 KB)	6.0	12.0	3.0	2.5
Git clone	14.0	28.0	10.0	8.0
Make xv6	3.5	6.0	2.5	2.0
Copy QEMU image	12.0	22.0	8.0	6.5
Ripgrep	8.0	14.0	5.0	4.0

2. **Compositional verification.** By combining relational specifications with rely-guarantee reasoning, CRL-H achieves modular proofs where each operation's correctness is established independently.

3. **Mechanized proofs.** All proofs are machine-checked in Coq, providing the highest standard of assurance.

4. **Practical performance.** AtomFS demonstrates that a verified concurrent file system can achieve usable performance with multicore scalability.

4.7 Limitations

1. **Enormous proof effort.** The 94:1 proof-to-code ratio—over 63,000 lines of proof for under 700 lines of C—makes this approach challenging to extend to production-scale file systems. Even for a research team, such an investment is substantial.

2. **In-memory design.** AtomFS operates in memory via FUSE, avoiding direct disk interaction. Crash safety and persistent-state reasoning, which FSCQ addressed, are not handled.

3. **Limited POSIX coverage.** Six interfaces are verified, and the system lacks many production features (extended attributes, ACLs, snapshots, quotas).

4. **Lock coupling trade-off.** The non-bypassable criterion and lock coupling, while amenable to verification, may limit concurrency in workloads with deep directory hierarchies or high contention on common path prefixes.

4.8 Position in the Evolution of File System Verification

CRL-H and AtomFS represent the logical extension of FSCQ's verification philosophy to the concurrent domain. The helper mechanism is a genuinely novel contribution that addresses a problem unique to file systems. However, the enormous proof effort also highlights the limits of purely manual verification—an observation that directly motivates the shift toward specification-guided generation in SysSpec. The reliance on Coq for both FSCQ and AtomFS establishes a common formal substrate, and the modular decomposition patterns developed in these works (layered invariants, rely-guarantee) inform the specification language design of SysSpec.

Chapter 5
SysSpec and SpecFS: A Case for Generative File Systems

5.1 Motivation and Problem Setting

The preceding two chapters demonstrate that formal verification can establish strong correctness guarantees for file systems, but at enormous cost in human effort: a 10:1 proof-to-code ratio for a sequential file system (FSCQ) and a 94:1 ratio for a concurrent one (AtomFS). Neither approach has been adopted by production file system developers. Meanwhile, large language models (LLMs) have shown remarkable abilities in code generation across domains, yet attempts to generate a complete, functional file system from natural language prompts have consistently failed.

Liu et al. [27] identify three fundamental challenges:

1. **Specification semantic gap.** Natural language cannot precisely capture the multi-faceted semantics of file systems. This includes functional correctness (POSIX behavior across all system calls), non-functional properties critical for performance (on-disk layout choices such as bitmap versus linear-scan block allocation, which affect I/O patterns), and complex concurrency control (lock ordering, deadlock avoidance, fine-grained access protocols). A prompt saying "implement a file system" provides no concrete semantics for any of these dimensions.

2. **Complex component composition.** LLMs' finite context windows preclude generating an entire file system monolithically. Modular generation introduces two sub-problems. First, when generating one module at a time, the LLM lacks global context about other (yet-to-be-generated or pre-existing) components, leading to interface mismatches. Second, during evolution, a local change can trigger cascading effects across multiple modules—a feature like extents modifies core data structures such as the inode, affecting every module that accesses inodes.

3. **Unreliable LLM capability.** LLM-based code generation is inherently nondeterministic due to hallucination. Identical specifications can yield different and potentially incorrect code outputs across generation attempts. For systems software, a naive "generate-and-pray" approach is unacceptable; a robust framework must incorporate validation to ensure generated code adheres to its specification.

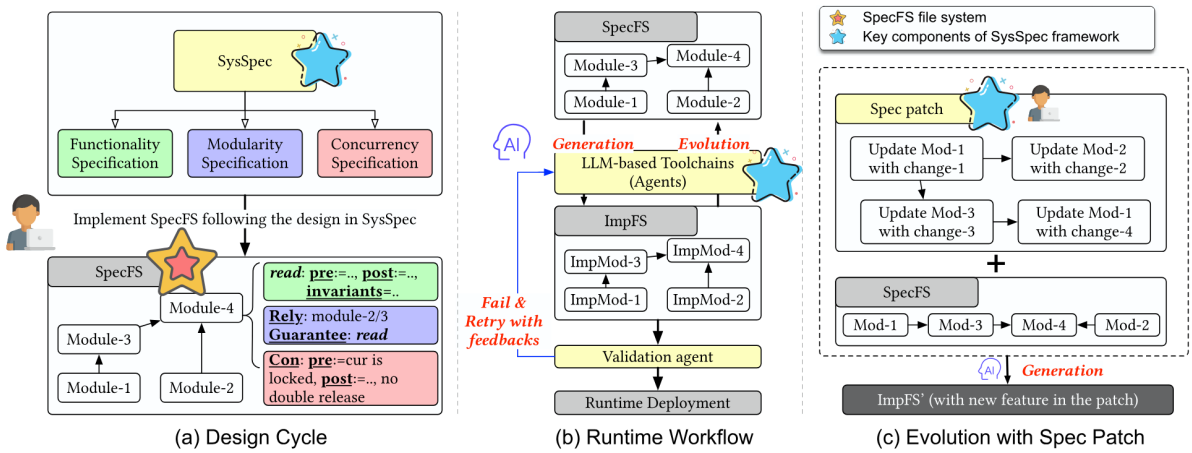

Figure 5.1: Design overview. (a) Developers write a file system (SpecFS) using a structured specification language that defines functionality, modularity, and concurrency. (b) An LLM-based toolchain generates a low-level implementation (ImpFS) from the specification and uses a validation agent to ensure correctness. (c) The file system evolves by applying a high-level spec patch to SpecFS, after which the toolchain regenerates the implementation to include the new features or fixes.

The central insight of SysSpec is to *replace ambiguous natural language with principles adapted from formal methods*. The resulting specification language serves as an unambiguous blueprint for LLM-based code generation while remaining more concise and maintainable than the generated code.

5.2 The SysSpec Specification Language

SysSpec decomposes a file system's specification into three orthogonal dimensions, as shown in Figure 5.1, each addressing one of the challenges above.

5.2.1 Functionality Specification

Drawing from Hoare logic, each function's behavior is specified using:

- **Pre- and post-conditions** that define contractual obligations. For `atomfs_ins(path, name)`, the precondition specifies that `path` is a NULL-terminated string array and `name` is a valid string. The postcondition describes two cases: successful traversal and insertion (new inode created, entry inserted into target directory, return 0) or failure (return -1).
- **Invariants** that must hold across all state transitions. For `atomfs_ins`, an example invariant is that `root_inum` always exists, ensuring the file system root is never deleted.

- **System algorithm** that outlines high-level logic. For complex operations like `atomfs_rename`, the algorithm is specified in three phases: (1) traversing the common path of source and destination, (2) traversing the remaining divergent paths, and (3) performing checks and executing the rename operation. The fine-grained locking scheme (which locks to acquire at each phase) is explicitly described.

- **Intent** that provides natural-language high-level guidance to steer the LLM toward better implementation choices, complementing the formal specifications. For example, when reading a large file extent, the developer can use intent to suggest a single bulk I/O operation; without this hint, the LLM might generate a correct but inefficient implementation that reads each disk block individually.

Not all components are required for every module. The authors identify three complexity levels: Level 1 modules (straightforward logic) need only pre/post-conditions and invariants; Level 2 modules (complex logic) additionally benefit from intent descriptions; Level 3 modules (highly optimized designs) require explicit algorithmic descriptions, as it is unreasonable to expect an LLM to derive optimized logic from scratch.

5.2.2 Modularity Specification

SysSpec decomposes the file system into modules with two constraints inspired by LLM reasoning properties. First, strict size constraints ensure each module ($\leq$500 lines of code) fits entirely within the model's context window, enabling holistic analysis during generation. Second, interface contracts govern inter-module dependencies through *Rely-Guarantee* conditions, adapted from concurrent program verification.

Where traditional rely conditions specify permissible environment interference for threads, SysSpec's `Rely` clauses enumerate a module's assumptions about other components. Similarly, `Guarantee` clauses replace thread behavior specifications with module interface contracts. Each module's Rely conditions must be entailed by the Guarantees of its dependencies, which ensures compositional correctness through localized synthesis.

In Figure 5.2, the module's Rely clause imports critical elements from its dependent modules, e.g., the definition of structures, lock/unlock primitives, path traversal function. The generated code then exports its own Guarantee, allowing dependents to build upon its functionality without needing to understand internal implementation details.

5.2.3 Concurrency Specification

A distinguishing feature of SysSpec is that it decouples concurrency logic from functional logic. The generation process uses two passes:

```
[RELY]
Predefined Structures/Functions:
struct inode { ... };
struct inode* root_inum;
void lock(struct inode*)
void unlock(struct inode*)
struct inode* locate(struct inode* cur, char*
↪ path[]) // Traverse path under cur
void insert(struct inode*, struct inode*,
↪ char*) // ...
int check_ins(struct inode*, char*) // ...

[GUARANTEE]
Exported Interface:
int atomfs_ins(char*[], char*, int,
               unsigned, unsigned)
```

Figure 5.2: Rely-Guarantee specifications for atomfs_ins. Both rely and guarantee conditions are simplified for clarity.

1. **First pass:** The LLM generates primary functional code from the functionality and modularity specifications, without considering concurrency.

2. **Second pass:** Using the dedicated concurrency specification, the LLM instruments the code with the required locking, synchronization, and atomicity behavior.

This separation is motivated by the observation that LLMs struggle to simultaneously satisfy functional and concurrency constraints when they are interleaved in a single prompt. The concurrency specification includes Rely clauses that capture locking requirements of internal functions. For instance, the concurrency specification for `atomfs_ins` specifies that `locate` requires its parameter inode to be locked as a precondition; since `atomfs_ins` itself has no such precondition, the generated code must acquire the lock on `root_inum` before invoking `locate(root_inum, path)`. The two-pass approach makes the synthesis task tractable by allowing the LLM to handle two distinct, complex problems sequentially.

5.2.4 DAG-Structured Specification Patches

To support evolution without breaking existing invariants, SysSpec introduces *DAG-structured specification patches*. Each patch is a self-contained description of a new feature, organized as a directed acyclic graph:

- **Leaf nodes** represent localized, self-contained changes within a single module. They introduce new logic and provide new Guarantees without affecting other parts of the system. A leaf node can also define new data structures or functions that subsequent nodes will rely on.

- **Intermediate nodes** build on the new Guarantees provided by their child nodes to implement more complex logic. In turn, they provide higher-level Guarantees, forming a dependency chain that progressively constructs the feature.
- **Root nodes** serve as the final integration point. Their specification provides a new Guarantee semantically equivalent to an existing interface, making the complete set of new functionality visible to the entire system.

For example, implementing extents (contiguous block ranges) transforms the file structure from individual blocks into extents. The modification process begins by altering low-level file operations (`lowlevel_file`) to incorporate extent-based I/O operations (leaf nodes). Subsequently, the `inode_management` module is modified to invoke these new extent operations (intermediate node). Finally, the root node integrates all changes, making extent-based operations the new default for the system.

5.3 The SysSpec Toolchain

SysSpec provides an LLM-based toolchain with four components:

SpecCompiler. The core translation engine from specification to C code. It employs a two-phase prompting strategy: first generating functional logic from the functionality and modularity specifications, then adding concurrency instrumentation. Internally, it contains two sub-agents:

- **CodeGen agent:** Generates C implementation from the specification.
- **SpecEval agent:** Reviews the generated code against the specification using a separate, stronger LLM instance. If flaws are detected, CodeGen is re-invoked with feedback. This validation loop exploits the fact that verifying a solution against rules is a simpler cognitive task than generating the solution from scratch, and that the probability of two distinct models making complementary errors on the same logic is extremely low.

SpecValidator. Performs final, holistic verification of the complete C implementation. It combines specification-based review (re-using SpecEval logic) with traditional software engineering workflows: running a suite of unit and regression tests against the generated file system. This emulates a CI/CD pipeline where the SpecEval component acts as an automated code reviewer while the test suite guards against regressions.

SpecAssistant. An interactive tool that streamlines specification development through a human-in-the-loop process. The developer provides a draft specification; SpecAssistant validates and reformats it to meet SysSpec syntax, then enters an automated refinement loop. It repeatedly invokes SpecCompiler; if the SpecEval phase identifies flaws, a

SpecFine step automatically polishes the specification based on the feedback before retrying. On success, the developer receives the refined specification and generated implementation. On failure, the last attempted specification is returned with detailed diagnostics serving as a debug log.

5.4 SpecFS: A Concrete Instantiation

SpecFS is the file system generated using SysSpec, based on the high-level design of AtomFS. Rather than porting AtomFS's C code, the authors undertook a complete reimplementation guided by SysSpec specifications, reusing some pre/post-conditions from AtomFS's formal Coq specifications.

SpecFS is organized into 45 distinct modules across several logical layers: file operations, inode management, path traversal, and POSIX interface. The modules are classified as 40 concurrency-agnostic modules and 5 thread-safe modules. SpecFS supports a comprehensive range of POSIX calls and comprises approximately 4,300 lines of generated C code. For context, this ranks 42nd among 82 file systems in the Linux 6.1.10 kernel by code size.

5.5 Evaluation

5.5.1 Generation Accuracy

The authors evaluated generation accuracy using two baselines on the 45 AtomFS modules:

- **Normal baseline:** LLM given only file correspondence logic and dependency module APIs (few-shot learning).
- **Oracle baseline:** Normal baseline plus access to ground-truth C code of dependency modules.

A module is considered correct if it passes all functional tests and is deemed logically equivalent to ground truth through manual inspection. Four LLMs of decreasing capability were tested: Gemini-2.5-Pro, DeepSeek-V3.1 Reasoning, GPT-5-minimal, and Qwen3-32B.

Results show that SysSpec consistently achieves higher generation accuracy than the normal baseline across all models. For Gemini-2.5-Pro, SysSpec generated 44 of 45 modules correctly. For DeepSeek-V3.1 Reasoning with the SpecValidator, all 45 modules were generated correctly—matching the oracle baseline.

5.5.2 Evolvability

The authors demonstrated SpecFS's evolvability by implementing 10 well-known Ext4 features through specification patches. These span four categories: (I) file structure modifications (indirect blocks, extents, inline data), (II) design updates for existing operations (multi-block pre-allocation, delayed allocation, rbtree for pre-allocation), (III) new functionalities (metadata checksums, encryption, logging/jbd2), and (IV) hyperparameter/metadata modifications (nanosecond timestamps). SysSpec consistently exhibited higher generation accuracy across all models compared to baselines.

Table 5.1: Case study of applying Ext4 features to SpecFS. Features span four categories: file structure modifications, design updates, new functionalities, and metadata modifications.

Type	**Feature**	**Propose**	**Launch**	**Description**	**Release**
I	Indirect Block (Ext2/3)	–	–	One-to-one block mapping via multi-level pointers	–
I	Extent	2006	2006	Contiguous block ranges reducing metadata by 50%	2.6.19
I	Inline Data	2011	2013	Store small files in inode's unused space	3.8
II	Multi-Block Pre-Allocation	2006	2008	Allocate blocks in groups for large files	2.6.25
II	Delayed Allocation	2006	2008	Deferred block allocation to reduce I/O	2.6.27
II	rbtree for Pre-Allocation	2022	2023	rbtree to organize pre-allocated block pool	6.4
III	Metadata Checksums	2011	2012	Checksummed filesystem metadata structures	3.5
III	Encryption	2015	2015	Per-directory encryption with low overhead	4.1
III	Logging (jbd2)	2006	2006	Journaling support for 64-bit filesystems	2.6.19
IV	Nanosecond Timestamps	2006	2006	Nanosecond resolution in inode structure	2.6.19

5.5.3 Ablation Study

An ablation study using DeepSeek-V3.1 Reasoning quantified the contribution of each specification component. For concurrency-agnostic modules, functionality specification alone achieved 30% accuracy (12/40), largely due to interface mismatches. Adding modularity specification (Rely-Guarantee) raised accuracy to 100% (40/40). For thread-safe modules, functionality alone and functionality-plus-modularity both achieved 0% (0/5). Adding concurrency specification raised this to 80% (4/5), and SpecValidator brought it to 100% (5/5). This confirms that all three specification dimensions are necessary for generating a correct concurrent file system.

Table 5.2: Ablation study quantifying the contribution of each specification component. Results use DeepSeek-V3.1 Reasoning. "Func", "Mod", and "Con" denote Functionality, Modularity, and Concurrency specifications respectively.

Modules	**Func**	**+Mod**	**+Con**	**+SpecValidator**
Concurrency-agnostic (40)	30% (12/40)	100% (40/40)	–	–
Thread-safe (5)	0% (0/5)	0% (0/5)	80% (4/5)	100% (5/5)

5.5.4 Productivity

SysSpec substantially improved programming productivity. Implementing the "extent" feature required 3.0× less effort with SpecFS compared to manual C coding in AtomFS. Implementing the "rename" module required 5.4× less effort. Across all modules, specification code is consistently shorter than the generated C code, confirming that specifications capture design intent more concisely than implementations.

Table 5.3: Productivity improvement of SpecFS over manual C development for implementing the extent feature and the rename module.

Development Task	**Manual**	**SpecFS**	**Improvement**
Extent feature	4.5 h	1.5 h	3.0×
Rename module	13 h	2.4 h	5.4×

5.5.5 Performance

Performance evaluations measured the impact of new features implemented by SpecFS, normalizing results before and after optimization. Test workloads included xv6 compilation (macro-benchmark), copying QEMU images, small-file operations (10K files at 1KB

each, metadata-intensive), and large-file operations (10MB file, data-intensive). New features such as extents and delayed allocation produced measurable performance improvements, confirming that specification-guided generation produces not only correct but also performant code.

5.6 Strengths

1. **Redefines the developer's role.** SysSpec shifts the developer from writing code to designing specifications. Specifications are shorter, more maintainable, and closer to design intent than the generated implementation.

2. **Practical evolvability.** The DAG-structured patch mechanism directly addresses the maintenance burden that dominates file system lifetimes (82.4% of Ext4 commits). Features can be added through specification patches rather than code patches, with the toolchain handling regeneration.

3. **Separation of concerns.** Decoupling functional, modular, and concurrency specifications enables LLMs to handle each concern independently, dramatically reducing generation errors. The two-pass concurrency approach is a pragmatic solution to LLM limitations.

4. **Quantified productivity gains.** The 3–5× productivity improvement, measured on real file system features, provides concrete evidence of the approach's practical value.

5.7 Limitations

1. **No machine-checked proofs.** Unlike FSCQ and AtomFS, SpecFS provides no formal correctness guarantee. Correctness is established through testing and LLM-based validation, which cannot rule out subtle concurrency bugs or corner cases.

2. **Dependence on LLM quality.** The approach inherits LLM non-determinism. While the agentic validation loop reduces errors, there is no guarantee that retrying will produce correct code for any given specification, and the validation itself is LLM-based and fallible.

3. **Specification effort.** Writing precise SysSpec specifications requires expertise in formal methods concepts (invariants, rely-guarantee). The framework lowers the barrier relative to Coq proofs, but does not eliminate the need for formal reasoning skill.

4. **Scalability to production systems.** SpecFS implements 4,300 lines of code—a fraction of Ext4's size. Whether the specification methodology scales to production-scale systems with tens of thousands of lines of code and complex cross-module interactions remains an open question.

5.8 Position in the Evolution of File System Development

SysSpec represents the culmination of the trajectory traced by this report. FSCQ proved that mechanized crash-safety verification is feasible. CRL-H proved that concurrent linearizability can be verified, at significant cost. SysSpec asks whether the precision of formal specifications can be retained while dramatically reducing human effort—not through better proof automation, but by redefining the developer's task from proof-writing to specification-writing, with LLMs bridging the gap to executable code. While the formal guarantees are weaker, the productivity gains and evolvability support suggest a pragmatically promising direction.

Chapter 6
Cross-Paper Synthesis and Open Challenges

6.1 Revisiting the Research Questions

6.1.1 Correctness Scope

The correctness guarantees provided by each system differ in kind and strength:

- **FSCQ** guarantees *crash safety* for a sequential file system: no data loss, structural consistency, and operation atomicity under arbitrary crash-reboot sequences. These guarantees are machine-checked (Coq) and exhaustive. They do not, however, address concurrency.
- **AtomFS** guarantees *linearizability* for a concurrent file system: every concurrent execution is equivalent to some sequential execution of atomic operations. Path inter-dependency is handled through the helper mechanism. The guarantees are also machine-checked in Coq. Crash safety is not separately addressed (the system is in-memory).
- **SpecFS** guarantees *specification conformance* validated through testing and LLM-based review: generated code passes regression tests and manual equivalence inspection. It additionally guarantees *evolvability* through DAG-structured patches. The guarantees are empirical rather than formal; no theorem connects the specification to the implementation.

There is a clear trade-off between guarantee strength and coverage breadth: the strongest guarantees apply to the narrowest systems (sequential, simplified), while the broadest coverage (many features, evolvable) comes with the weakest guarantees.

6.1.2 Role of Formal Methods

The role of formal methods shifts across the three papers:

- In FSCQ, formal methods are a **post-design verification tool**. CHL is embedded in Coq; the developer designs, implements, and proves correctness as separate activities. The proof is a separate artifact from the design.

Table 6.1: Cross-paper comparison of proof effort, performance, and generality across FSCQ, AtomFS, and SpecFS.

Dimension	**FSCQ**	**AtomFS**	**SpecFS**
Proof-to-code ratio	∼10:1	∼94:1	N/A (spec-to-code ≪1:1)
Guarantee type	Machine-checked	Machine-checked	Empirical (tests + LLM review)
Concurrency	Sequential	Fine-grained	Fine-grained
Performance overhead	∼4× CPU (Haskell extraction)	Comparable to big-lock	Comparable to manual C
Feature scope	xv6 subset	6 POSIX calls	45 modules, broad POSIX
Evolvability	Modular (3 case studies)	Not evaluated	10 Ext4 features demonstrated

- In CRL-H, formal methods are a **co-design framework**. The logic's requirements—the non-bypassable criterion, the helper mechanism's structure, the form of invariants—directly shape how AtomFS is architected (lock coupling, ghost state layout). The proof and the design co-evolve.
- In SysSpec, formal-method-inspired constructs are the **primary design artifact**. The specification *is* the design; implementation is derived by the toolchain. The developer reasons at the specification level; the LLM handles the translation to C.

This trajectory reflects a broader trend in software engineering: the center of gravity shifts from implementation to specification as automation improves.

6.1.3 Proof Effort, Performance, and Generality

The three papers embody very different points in a multi-dimensional trade-off space:

No single approach dominates. FSCQ provides the strongest sequential guarantees with moderate effort. AtomFS extends guarantees to concurrency at high cost. SpecFS achieves the broadest feature coverage and best evolvability at the cost of formal certainty.

6.2 Open Challenges

Several open problems emerge from this comparative analysis:

1. **Unifying proof and generation.** The long-term vision—combining the productivity of LLM-based generation with the certainty of mechanized proof—remains open. If an LLM could generate not only code but also machine-checkable proofs from specifications, it would combine the strengths of the SysSpec and FSCQ/AtomFS approaches. This requires advances in both LLM reasoning for formal mathematics and integration between generation toolchains and proof assistants.

2. **Specification language scalability.** SysSpec's specification language is designed for a file system of AtomFS's complexity. Whether similar languages can be designed for other systems domains (databases, network protocols, distributed consensus) and whether they scale to production-scale systems is unclear.

3. **LLM reliability for systems code.** All three papers ultimately depend on human expertise: FSCQ and AtomFS for writing proofs, SysSpec for writing specifications. Can LLMs be made *provably* faithful to formal specifications through constrained decoding, neuro-symbolic approaches, or formal verification of the generation process itself?

4. **Crash safety for concurrent file systems.** No single paper in this survey simultaneously addresses crash safety and concurrency. DFSCQ [4] and SFSCQ [3] make progress in this direction, but a fully verified concurrent, crash-safe file system remains an open challenge.

5. **From specification to optimizations.** SysSpec's specifications capture functional correctness and concurrency, but performance-critical optimizations (e.g., specific block allocation policies, read-ahead heuristics) are specified through intent and system algorithms. The extent to which LLMs can autonomously discover and implement effective optimizations while preserving correctness guarantees is an open question.

Chapter 7
Conclusion and Future Work

This report has examined three landmark papers in file system correctness spanning a decade of research: FSCQ (SOSP 2015), CRL-H with AtomFS (SOSP 2019), and SysSpec with SpecFS (2025). Each addresses a distinct facet of the file system development challenge, and together they trace a coherent trajectory in how the systems community approaches correctness.

FSCQ demonstrated that end-to-end verification of crash safety is feasible for a real file system. Through Crash Hoare Logic's innovations—crash conditions, recovery procedures, logical address spaces, and proof automation—it produced the first file system with a machine-checkable proof that it correctly recovers from any sequence of crashes. The 10:1 proof-to-code ratio, while substantial, was shown to be manageable for a small research team and amenable to incremental modification.

CRL-H extended formal guarantees to the concurrent setting by addressing the fundamental challenge of path inter-dependency. Its helper mechanism, ghost state framework, and roll-back procedure provide a general solution to the external linearization point problem that affects all surveyed concurrent file systems. AtomFS, the resulting verified concurrent file system, achieves scalable performance on multicore processors. The 94:1 proof-to-code ratio, however, underscores the enormous cost of concurrent verification and motivates the search for more efficient approaches.

SysSpec proposed a paradigm shift: rather than writing code and proving it correct, developers write specifications and derive code from them. By replacing ambiguous natural language with formal-method-inspired constructs (Hoare-logic pre/post-conditions, rely-guarantee contracts, invariants, and concurrency specifications), SysSpec provides a sufficiently precise blueprint for LLMs to generate correct, evolvable file system code. SpecFS, the resulting system, passes hundreds of regression tests, implements 10 Ext4 features through DAG-structured patches, and achieves 3–5× productivity improvements over manual development.

The trajectory from proving to generating reflects a broader shift in systems engineering: as the cost of formal verification becomes prohibitive, the focus moves upstream to specification design, with automated toolchains handling the translation to implementation. This does not render formal methods obsolete; rather, it changes their role from a verification technique to a design discipline. The open challenge that animates future work is to close the gap between these approaches—to combine the productivity of LLM-based generation with the certainty of mechanized proof.

Bibliography

[1] J. Bonwick, M. Ahrens, V. Henson, M. Maybee, and M. Shellenbaum. The Zettabyte File System. *Sun Microsystems*, 2003.

[2] J. Bornholt, A. Amit, N. Amit, and M. Seltzer. Specifying and Checking File System Crash-Consistency Models. In *Proceedings of the 21st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '16)*, 83-98, 2016.

[3] T. Chajed. Verifying a Concurrent, Crash-Safe File System with Sequential Reasoning. *MIT Ph.D. Dissertation*, 2022.

[4] H. Chen, T. Chajed, A. Konradi, S. Wang, A. İleri, A. Chlipala, M. F. Kaashoek, and N. Zeldovich. Verifying a High-Performance Crash-Safe File System Using a Tree Specification. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP '17)*, 270-286, 2017.

[5] H. Chen, D. Ziegler, T. Chajed, A. Chlipala, M. F. Kaashoek, and N. Zeldovich. Using Crash Hoare Logic for Certifying the FSCQ File System. In *Proceedings of the 25th ACM Symposium on Operating Systems Principles (SOSP '15)*, 18-37, 2015.

[6] H. Chen, D. Ziegler, A. Chlipala, M. F. Kaashoek, and N. Zeldovich. Specifying Crash Safety for Storage Systems. In *Proceedings of the 15th USENIX Workshop on Hot Topics in Operating Systems (HotOS '15)*, 21, 2015.

[7] M. Chen, J. Tworek, H. Jun, Q. Yuan, et al. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*, 2021.

[8] X. Chen, M. Lin, N. Schärli, and D. Zhou. Teaching Large Language Models to Self-Debug. *arXiv preprint arXiv:2304.05128*, 2023.

[9] The Coq Development Team. The Coq Proof Assistant. `https://coq.inria.fr`.

[10] R. Cox, M. F. Kaashoek, and R. Morris. xv6: A Simple, Unix-like Teaching Operating System. *MIT CSAIL*, 2011.

[11] J. Derrick, G. Schellhorn, and H. Wehrheim. Mechanically verified proof obligations for linearizability. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 33(1):4, 1-43, 2011.

[12] ext4(5) - linux manual page. `https://man7.org/linux/man-pages/man5/ext4.5.html`, 2025.

[13] I. Filipović, P. O'Hearn, N. Rinetzky, and H. Yang. Abstraction for Concurrent Objects. In *Theoretical Computer Science*, 411(51-52):4379-4398, 2010.

[14] C. Hawk, H. Chen, M. F. Kaashoek, and N. Zeldovich. IronFleet: Proving Practical Distributed Systems Correct. In *Proceedings of the 25th ACM Symposium on Operating Systems Principles (SOSP '15)*, 1-17, 2015.

[15] M. P. Herlihy and J. M. Wing. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990.

[16] C. A. R. Hoare. An Axiomatic Basis for Computer Programming. *Communications of the ACM*, 12(10):576–580, 1969.

[17] C. B. Jones. Tentative Steps Toward a Development Method for Interfering Programs. *ACM Transactions on Programming Languages and Systems*, 5(4):596–619, 1983.

[18] R. Jung, D. Swasey, F. Sieczkowski, K. Svendsen, A. Turrini, L. Birkedal, and D. Dreyer. Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning. In *the 42nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '15)*, 2015.

[19] S. Kim, M. Xu, S. Kashyap, J. Yoon, W. Xu, and T. Kim. Finding semantic bugs in file systems with an extensible fuzzing framework. *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP '19)*, 147-161, 2019.

[20] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood. seL4: Formal Verification of an OS Kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP '09)*, 207-220, 2009.

[21] X. Leroy. Formal Verification of a Realistic Compiler. *Communications of the ACM*, 52(7):107–115, 2009.

[22] K. Levin, N. van Kempen, E. D. Berger, and S. N. Freund. ChatDBG: An AI-Powered Debugging Assistant. *arXiv preprint arXiv:2403.16354v2*, 2024.

[23] R. Li, L. Ben Allal, Y. Zi, N. Muennighoff, et al. StarCoder: May the Source Be With You! *arXiv preprint arXiv:2305.06161*, 2023.

[24] Y. Li, D. Choi, J. Chung, N. Kushman, et al. Competition-Level Code Generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

[25] H. Liang and X. Feng. Modular verification of linearizability with non-fixed linearization points. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '13)*, 459-470, 2013.

[26] H. Liang, X. Feng, and M. Fu. A Rely-Guarantee-Based Simulation for Verifying Concurrent Program Transformations. In *ACM SIGPLAN Notices*, 47(1):455-468, 2012.

[27] Q. Liu, M. Zou, H. Zhang, D. Du, Y. Xia, and H. Chen. Sharpen the Spec, Cut the Code: A Case for Generative File System with SysSpec. *arXiv preprint arXiv:2512.13047*, 2025.

[28] L. Lu, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau, and S. Lu. A Study of Linux File System Evolution. In *ACM Transactions on Storage (TOS)*, 10(1):3, 1-32, 2014.

[29] C. Min, S. Kashyap, B. Lee, C. Song, and T. Kim. Understanding Manycore Scalability of File Systems. In *Proceedings of the 2016 USENIX Annual Technical Conference (ATC '16)*, 71-85, 2016.

[30] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, et al. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. *arXiv preprint arXiv:2203.13474*, 2022.

[31] OpenAI. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*, 2023.

[32] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswany, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau. All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI '14)*, 433-448, 2014.

[33] J. C. Reynolds. Separation Logic: A Logic for Shared Mutable Data Structures. In *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science (LICS '02)*, 55-74, 2002.

[34] O. Rodeh, J. Bacik, and C. Mason. BTRFS: The Linux B-Tree Filesystem. *ACM Transactions on Storage*, 9(3):1–32, 2013.

[35] A. Sweeney, D. Doucette, W. Hu, C. Anderson, M. Nishimoto, and G. Peck. Scalability in the XFS File System. In *Proceedings of the 1996 annual conference on USENIX Annual Technical Conference (ATEC '96)*, 1, 1996.

[36] D. Woos, J. R. Wilcox, S. Anton, Z. Tatlock, M. D. Ernst, and T. Anderson. Planning for Change in a Formal Verification of the Raft Consensus Protocol. In *Proceedings of the 5th ACM SIGPLAN Conference on Certified Programs and Proofs (CPP '16)*, 154-165, 2016.

[37] J. Yang, C. E. Jimenez, A. Wettig, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[38] J. Yang, C. Sar, and D. Engler. eXplode: A Lightweight, General System for Finding Serious Storage System Errors. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)*, Seattle, WA, USA, November 2006.

[39] J. Yang, P. Twohey, D. Engler, and M. Musuvathi. Using Model Checking to Find Serious File System Errors. In *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI '04)*, San Francisco, CA, USA, December 2004.

[40] M. Zou, H. Ding, D. Du, M. Fu, R. Gu, and H. Chen. Using Concurrent Relational Logic with Helpers for Verifying the AtomFS File System. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP '19)*, 259-274, 2019.